

Sustainable Software Engineering - Total Cost Ownership & Technical Debt



**ECONOMIC
SUSTAINABILITY**



Economic Sustainability

Economic sustainability refers to the ability of an economy, organization, or system to support long-term growth without compromising social, environmental, or financial stability. It emphasizes:

- **Efficient resource use:** Minimizing waste and enhancing value.
- **Long-term viability:** Making decisions that do not threaten future generations.
- **Balanced cost-benefit:** Ensuring investments deliver sustainable returns.

TOTAL COST OF OWNERSHIP



Total Cost of Ownership

When choosing among alternatives in a purchasing decision, buyers often focus only on the **purchase price**, i.e., the short-term price. Nevertheless, there are also **operational costs**, which are long-term expenses incurred during the useful life and eventual disposal of the product.

The **total cost of ownership (TCO)** is the price to **purchase an asset** plus the **operating costs**. Assessing TCO involves evaluating the overall value of a product over time. The asset with the lower total cost of ownership can provide better value in the long run.

Direct expenses can be easily reported, whereas listing all indirect expenses is more complex, although it can significantly influence whether to complete a purchase.

Companies use TCO as a framework to analyze business deals, aiming to take a more holistic approach that assesses the purchase from a broad perspective. This analysis includes the initial purchase price and all direct and indirect expenses.

TCO and Economic Sustainability

TCO is a practical tool for achieving economic sustainability because it requires decision-makers to evaluate the true cost of ownership over time, aligning financial choices with long-term viability.

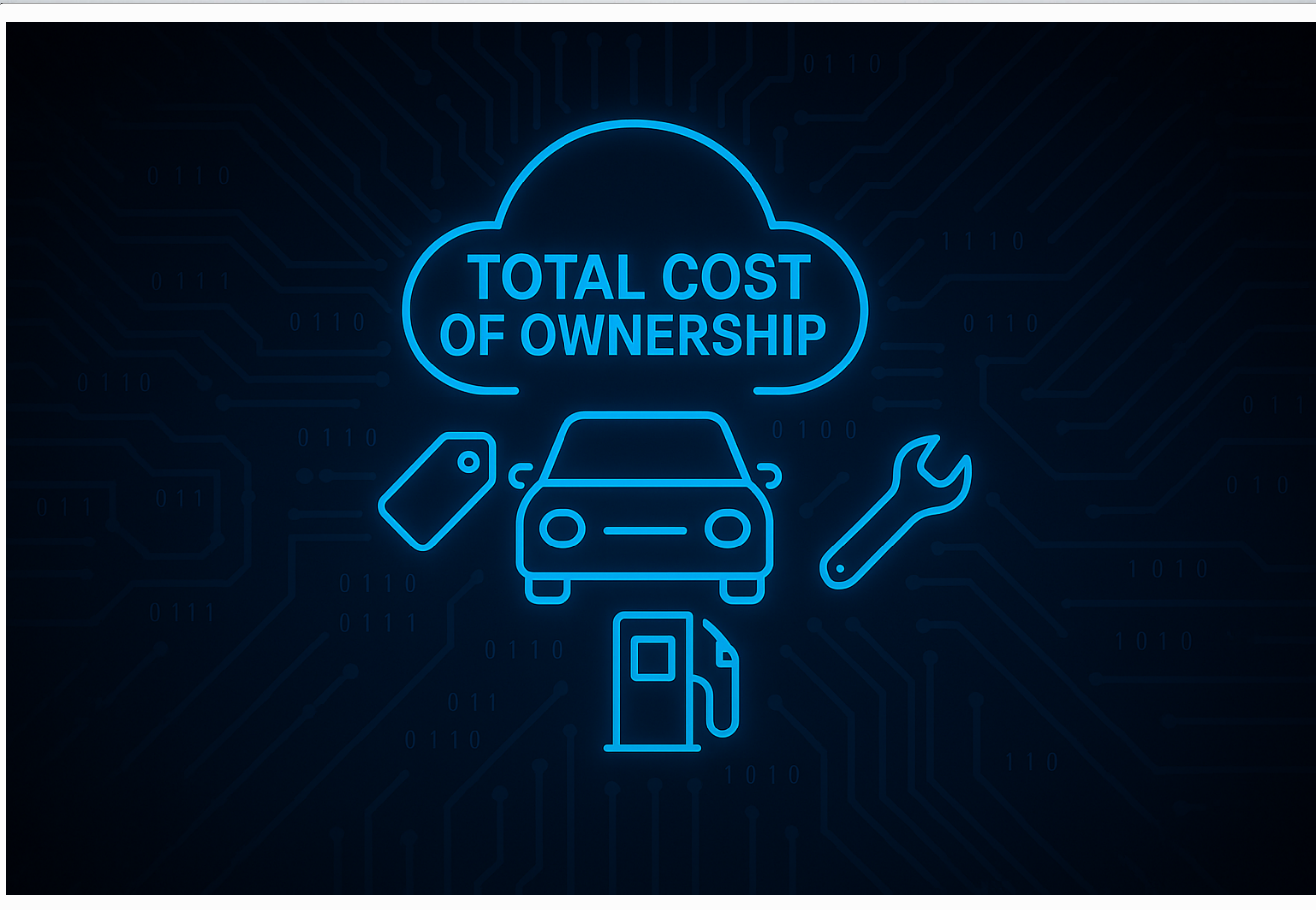
TCO and economic sustainability are closely related because:

- **Lifecycle Perspective:** TCO considers all costs throughout the product or service lifecycle (acquisition, operation, maintenance, disposal). Economic sustainability demands long-term cost efficiency, not just short-term savings.
- **Avoiding Hidden Costs:** Sustainable decisions aim to minimize hidden costs such as excessive maintenance, energy use, or early replacement. TCO analysis reveals these costs, helping organizations select economically sustainable options.
- **Investment vs. Operational Efficiency:** A low initial cost might lead to higher operational expenses, harming sustainability. TCO promotes strategic investments that lower long-term expenses and resource use.
- **Risk Management:** Poor TCO planning can cause financial instability (e.g., unexpected maintenance costs). Economic sustainability requires predictable, manageable costs.

Categories in the Total Cost of Ownership

Cost Category	Included Expenses	Description
Acquisition	Purchase price, licensing fees, shipping, installation, initial training	One-time costs to acquire and set up the asset or system.
Operation	Energy consumption, staffing, consumables, hosting/cloud fees	Day-to-day costs for running and using the asset effectively.
Ownership	Maintenance, upgrades, support contracts, insurance, compliance	Ongoing costs of keeping the asset functional and compliant over its lifecycle.
End-of-Life	Disposal, recycling, data migration, decommissioning	Costs incurred when retiring or replacing the asset, including compliance and transition efforts.

What is the TCO of this car?



What is the TCO of this car?



Category	Included Expenses	Description
Acquisition	Purchase price, registration, eco-tax, initial fees	€240,000 + €2,500 EcoTassa
Operation	Fuel, insurance, road tax, superbollo	€45,000 – €55,000
Ownership	Maintenance, repairs	€15,000 – €20,000
End-of-Life	Disposal, resale handling	€1,000 – €2,000

What about Software? Let's play a game.



Linux



LibreOffice

VS



Windows



MS Office

What about Software? Let's play a game.

You have 15,000 desktop machines. You need to choose between

- keeping Windows and MS Office
- migrating to Ubuntu and LibreOffice

considering TCO over 10 years.

Cost Category	Windows + MS Office	Linux + LibreOffice
Acquisition		
Operation		
Ownership		
End-of-Life		

What about Software? Let's play a game.

You have 15,000 desktop machines. You need to choose between

- keeping Windows and MS Office
- migrating to Ubuntu and LibreOffice

considering TCO over 10 years.

Cost Category	Windows + MS Office	Linux + LibreOffice
Acquisition	Windows licenses (€100–150 per PC), MS Office licenses (€250–350 per PC)	Linux OS (€0), LibreOffice (€0)
Operation		
Ownership		
End-of-Life		

What about Software? Let's play a game.

You have 15,000 desktop machines. You need to choose between

- keeping Windows and MS Office
- migrating to Ubuntu and LibreOffice

considering TCO over 10 years.

Cost Category	Windows + MS Office	Linux + LibreOffice
Acquisition	Windows licenses (€100–150 per PC), MS Office licenses (€250–350 per PC)	Linux OS (€0), LibreOffice (€0)
Operation	Hardware (300 € per PC)	Hardware (150€ per PC)
Ownership		
End-of-Life		

What about Software? Let's play a game.

You have 15,000 desktop machines. You need to choose between

- keeping Windows and MS Office
- migrating to Ubuntu and LibreOffice

considering TCO over 10 years.

Cost Category	Windows + MS Office	Linux + LibreOffice
Acquisition	Windows licenses (€100–150 per PC), MS Office licenses (€250–350 per PC)	Linux OS (€0), LibreOffice (€0)
Operation	Hardware (300 € per PC)	Hardware (150€ per PC)
Ownership	Support contracts, patching, proprietary updates (300€ per PC)	Internal support and training (250€ per PC).
End-of-Life		

What about Software? Let's play a game.

You have 15,000 desktop machines. You need to choose between

- keeping Windows and MS Office
- migrating to Ubuntu and LibreOffice

considering TCO over 10 years.

Cost Category	Windows + MS Office	Linux + LibreOffice
Acquisition	Windows licenses (€100–150 per PC), MS Office licenses (€250–350 per PC)	Linux OS (€0), LibreOffice (€0)
Operation	Hardware (300 € per PC)	Hardware (150€ per PC)
Ownership	Support contracts, patching, proprietary updates (300€ per PC)	Internal support and training (250€ per PC).
End-of-Life	Migration to newer Windows versions (200 € per PC)	LibreOffice updates (50€ per PC)

What about Software?



What about Software?

In 2004, Munich launched the LiMux project to migrate from Windows and MS Office to Linux and OpenOffice/LibreOffice across about 15,000 desktops. The goal was to reduce licensing costs, increase independence, and improve security. The migration was completed by 2012.

In 2017, Munich decided to revert to Windows, with all systems being replaced by Windows 10. Political reasons mainly drove the choice.

In 2020, a new political coalition decided to go back to Linux and LibreOffice.

Cost Category	Windows + MS Office	Linux + LibreOffice
Acquisition	Windows licenses (€100–150 per PC), MS Office licenses (€250–350 per PC) → ~€6–7M	Linux OS (€0), LibreOffice (€0) → €0
Operation	Higher hardware requirements, frequent upgrades → €3–4M	Lower hardware requirements → €1–2M
Ownership	Support contracts, patching, proprietary updates → €4–5M	Internal support, training → €3–4M
End-of-Life	Migration to newer Windows versions → €2–3M	LibreOffice updates → €1M
Total Cost	~€15–19M	~€4–7M



<https://en.wikipedia.org/wiki/LiMux>

What if we ignore TCO and focus on short-term solutions during development?

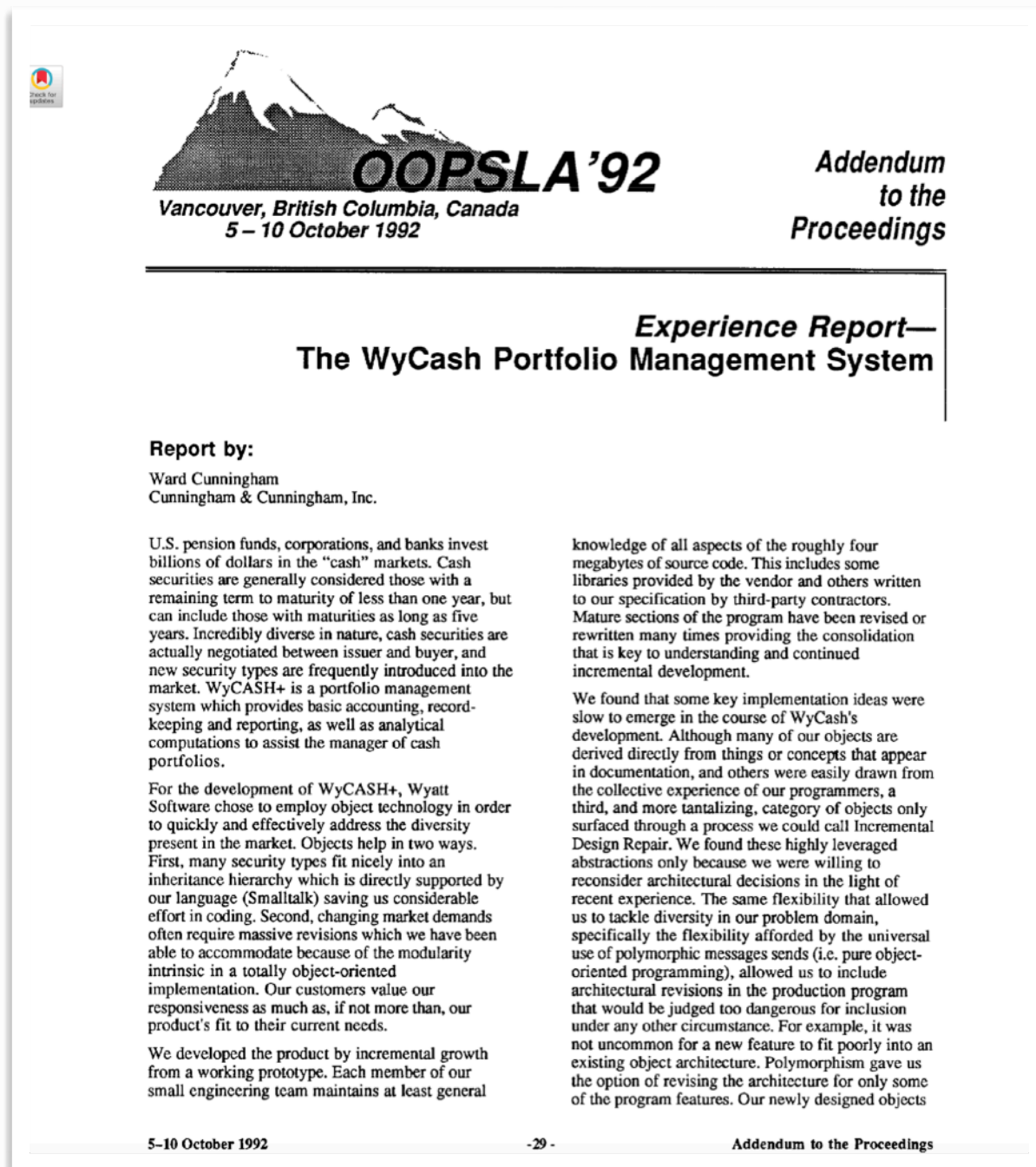


**TECHNICAL
DEBT**



What is Technical Debt?

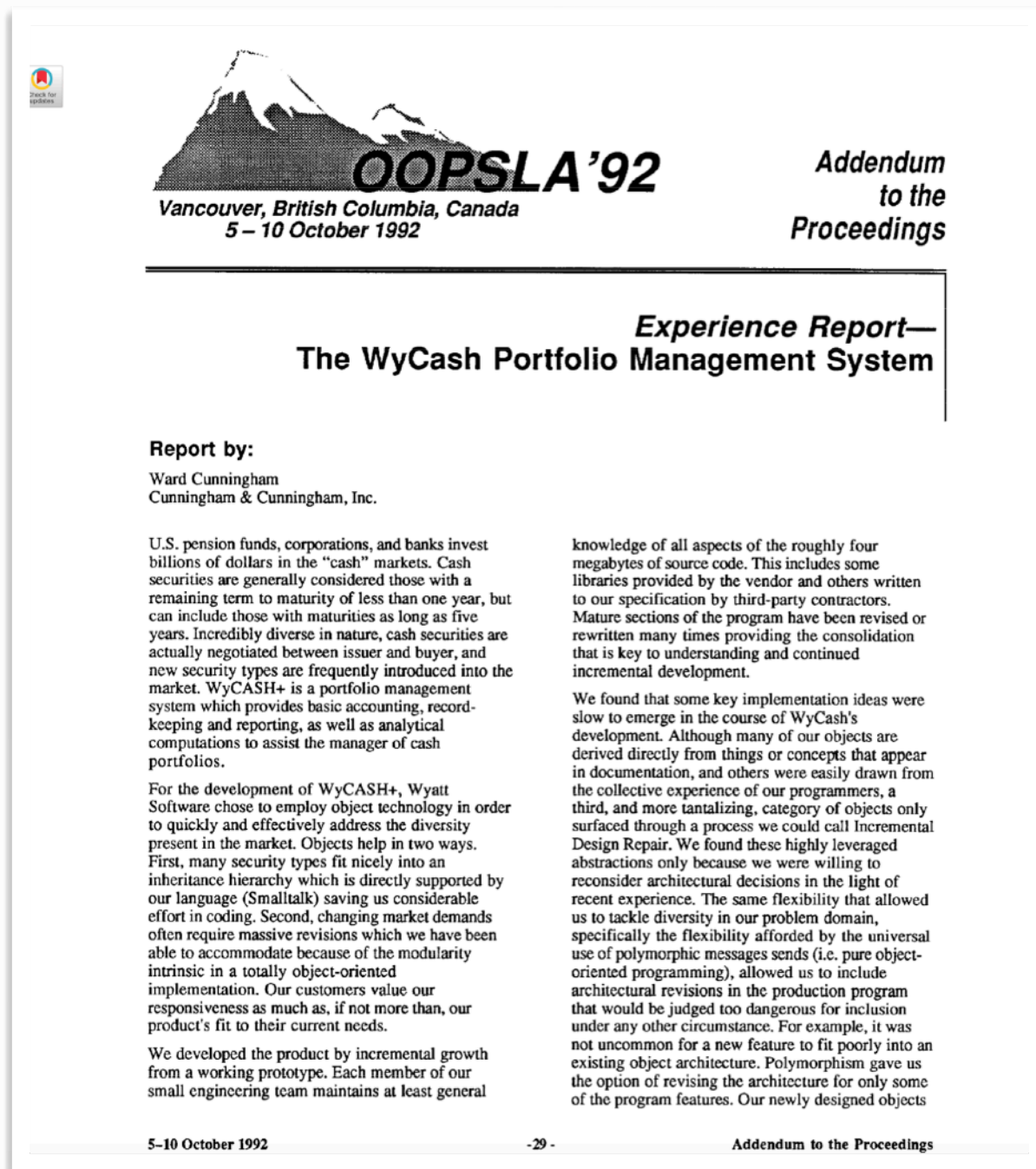
Technical Debt is a metaphor proposed by Ward Cunningham to describe consequences that can arise in a project [1].



[1] Cunningham, Ward. "The WyCash portfolio management system." *ACM Sigplan Ops Messenger* 4.2 (1992): 29-30.

What is Technical Debt?

Technical Debt is a metaphor proposed by Ward Cunningham to describe consequences that can arise in a project [1].

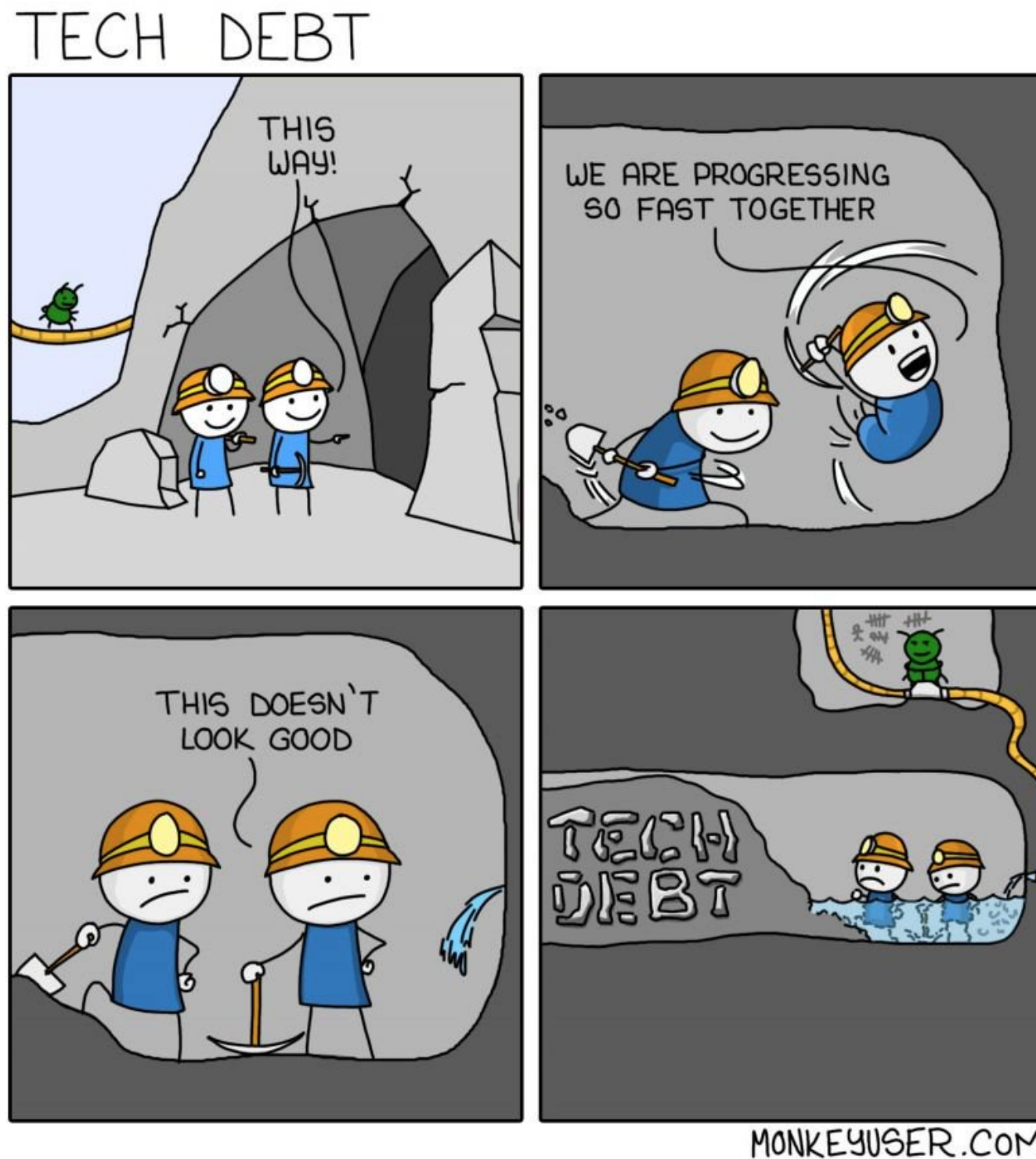


While this term is trendy and cool in these days, this term comes from 1992!

[1] Cunningham, Ward. "The WyCash portfolio management system." *ACM Sigplan Ops Messenger* 4.2 (1992): 29-30.

What is Technical Debt?

Technical debt is code written yesterday that's a burden today.



[1] Cunningham, Ward. "The WyCash portfolio management system." *ACM Sigplan Ops Messenger* 4.2 (1992): 29-30.

What is Technical Debt?

Technical Debt is a metaphor proposed by Ward Cunningham to describe the possible complications that arise in a project [1].

Intentional Technical Debt: When an organization, in order to address issues such as time-to-market or business constraints, makes a conscious decision to use a solution that brings benefits in the present rather than benefits in the future.

Unintentional Technical Debt: When one or more members of the organization accidentally implement a suboptimal solution that may lead to serious disadvantages in the future.

Environmental Tech Debt: When we do nothing about code, we accumulate tech debt without realizing it. The continuous evolution of software, libraries, and other elements in the world leads to our code becoming obsolete.

[1] Cunningham, Ward. "The WyCash portfolio management system." *ACM Sigplan Oops Messenger* 4.2 (1992): 29-30.

What is Technical Debt?

Technical debt is code written yesterday that's a burden today.

Reckless & Intentional

"We don't have time for design."

Prudent & Intentional

"We must ship now, deal with consequences later."

Reckless & Inadvertent

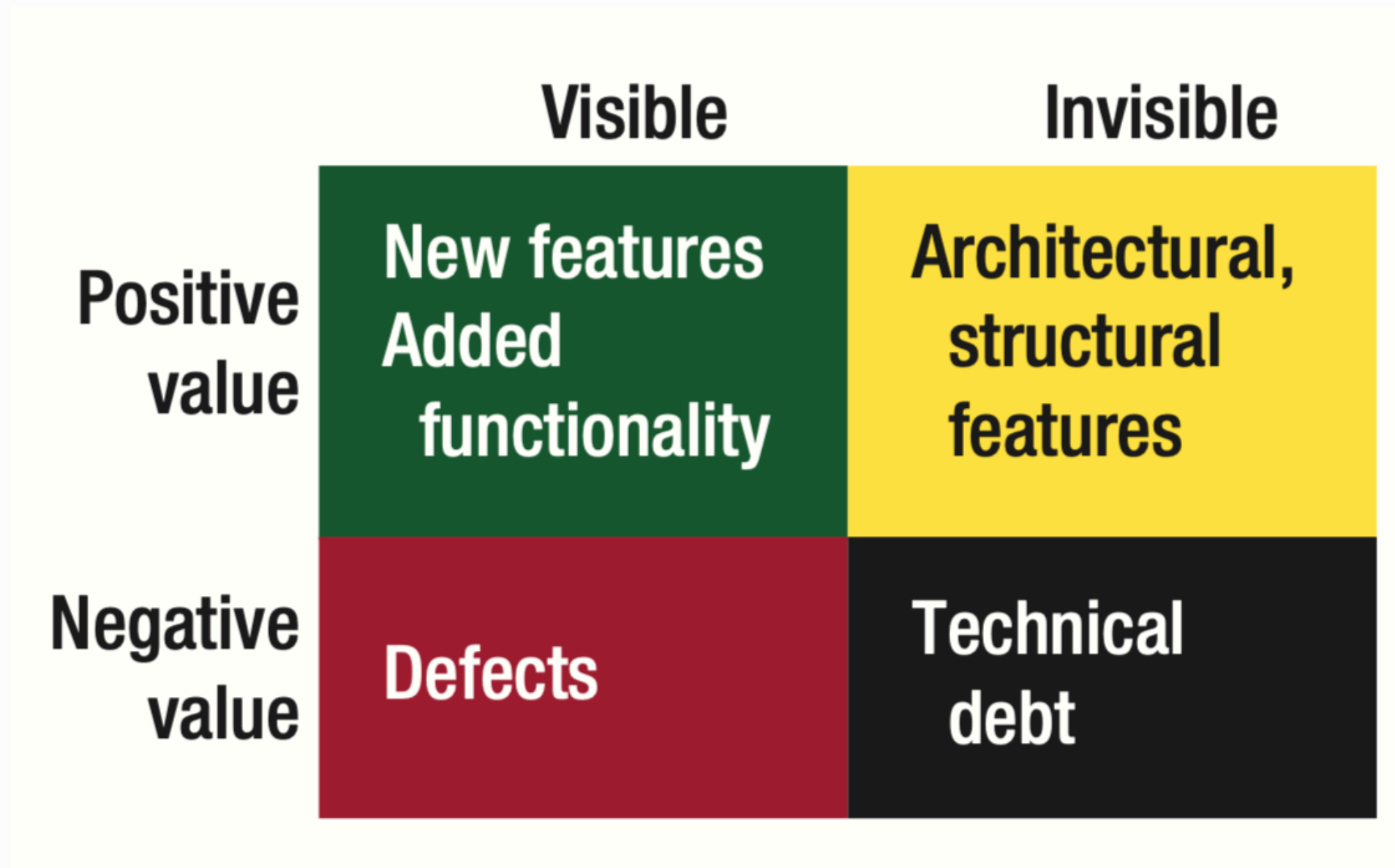
"What's layering?"

Prudent & Inadvertent

"Now we know how we should have done it."

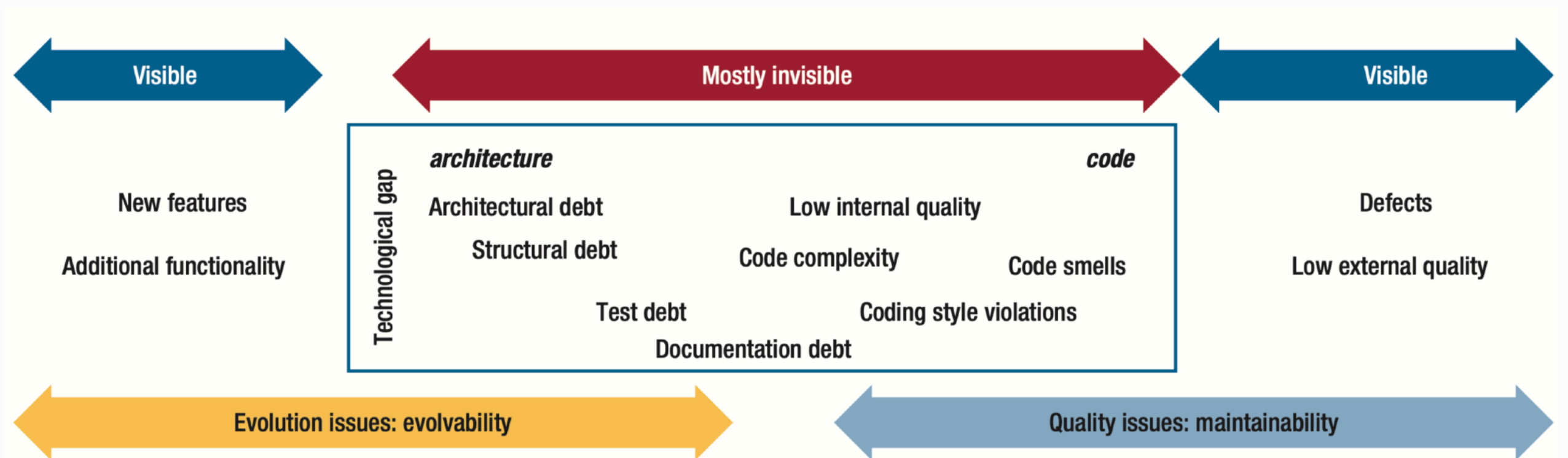
What is Technical Debt?

Technical debt is code written yesterday that's a burden today.



What is Technical Debt?

Technical debt is code written yesterday that's a burden today.



Why Technical Debt?

Technical debt is a matter of **money**.

The **Knight Capital Group** was an American global financial services firm engaging in **market making** and trading.

The Knight logo, featuring the word "Knight" in a blue, sans-serif font with a small registered trademark symbol.

On August 1, 2012, Knight Capital Group experienced **a major failure** in the operation of its automated routing system.

Under pressure, the team reused obsolete code that had not been properly decommissioned.

We will resolve that later... right?



Why Technical Debt?

Technical debt is a matter of **money**.

That obsolete code produced in only 45 minutes 7 billion erroneous trades...

Knight

Knight Capital, in 45 minutes, lost **the 70% percent of values of their stocks**.



Debt Price: 440.000.000 \$

Why Technical Debt?

Technical debt is a matter of **money**.

THE ECONOMIC IMPACT OF 'BAD CODE'

Sources: Evans Data Corp.,
CIA Factbook, Stripe research

41.1

Average hours per developer workweek

17.3

Average hours spent by developers on bad code, debugging, refactoring, modifying

13.5

Average hours spent on technical debt

3.8

Average hours spent on bad code

9.25%

Percent productivity loss from bad code

~\$85 billion

Global GDP loss from developer time spent on bad code annually

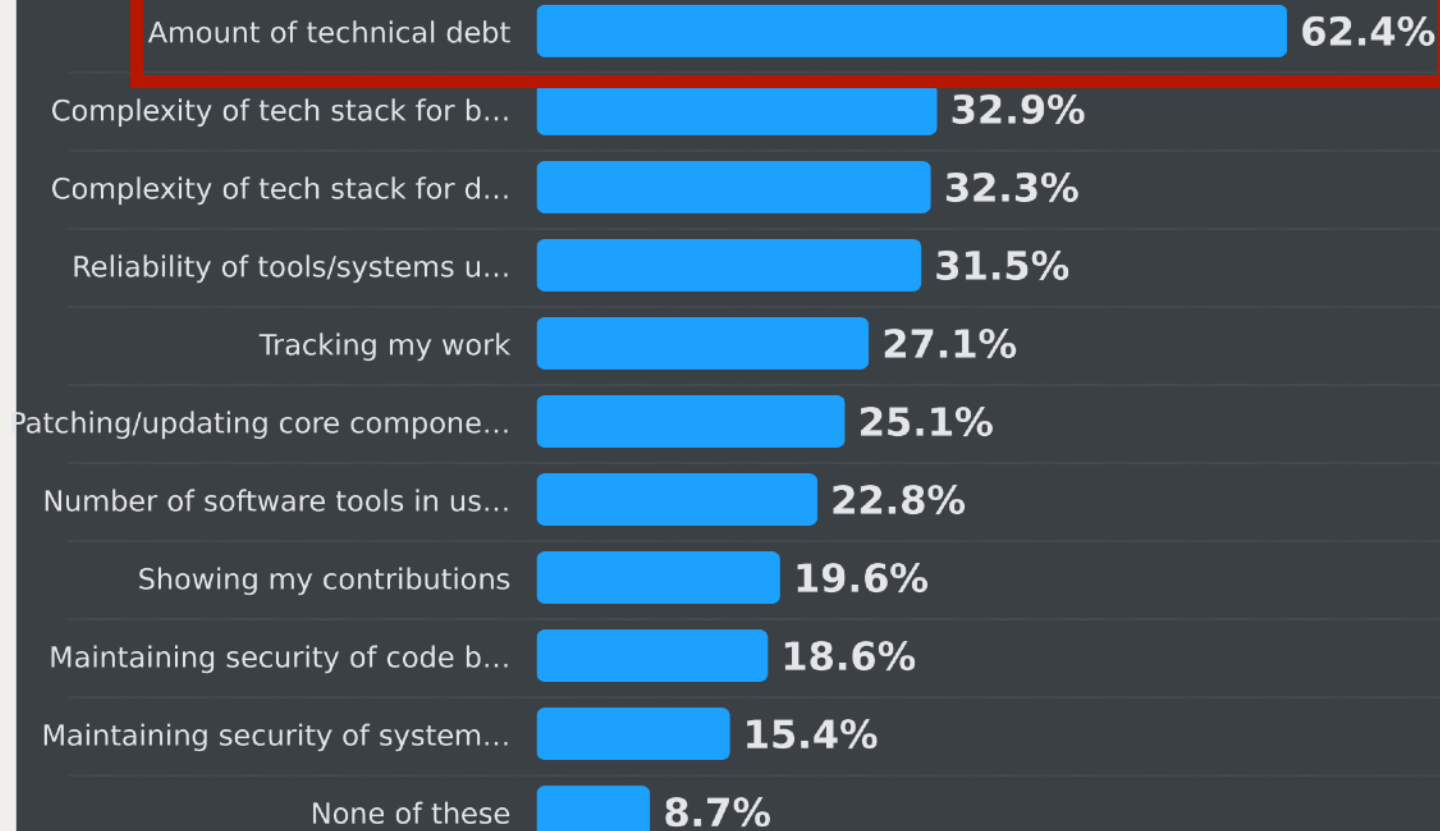
Why Technical Debt?

Technical debt is a matter of **people**.

StackOverflow developers survey reports Technical Debt as the most common cause of **developer's frustration**.

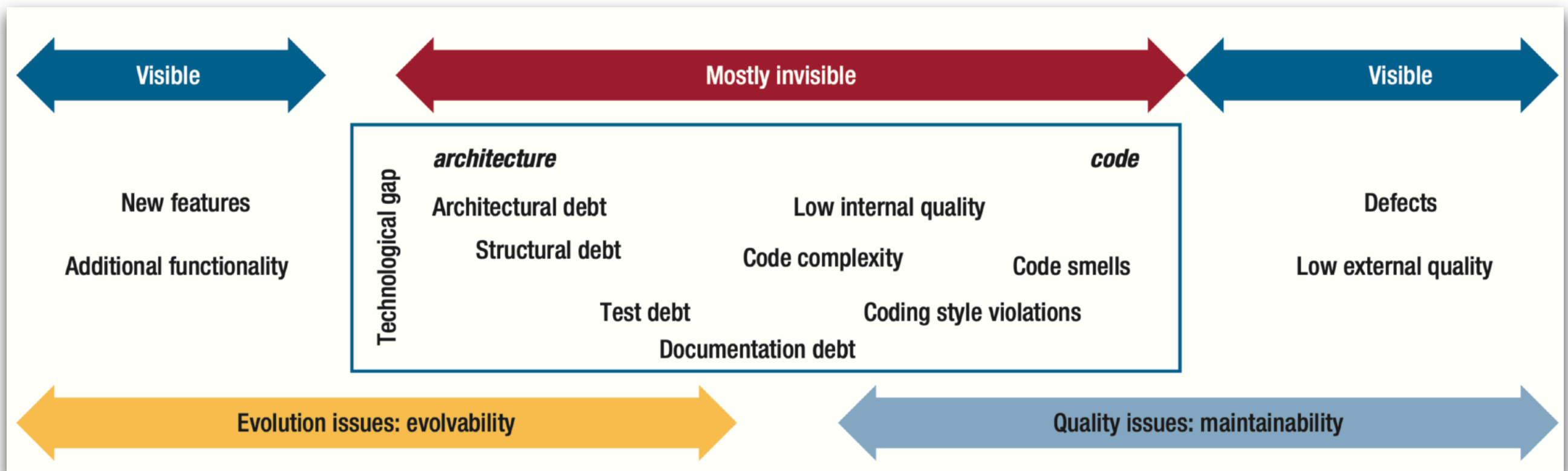
Developer Experience / All Respondents

Most common frustrations



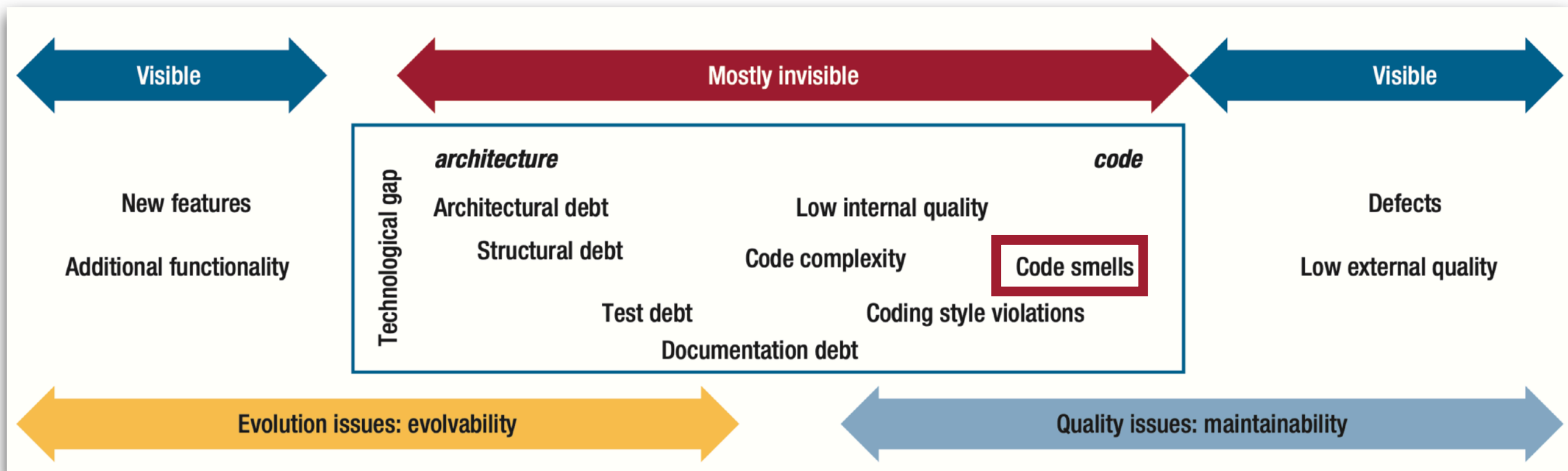
How can we manage Technical Debt?

How can we identify Tech Debt?



How can we manage Technical Debt?

How can we identify Tech Debt?



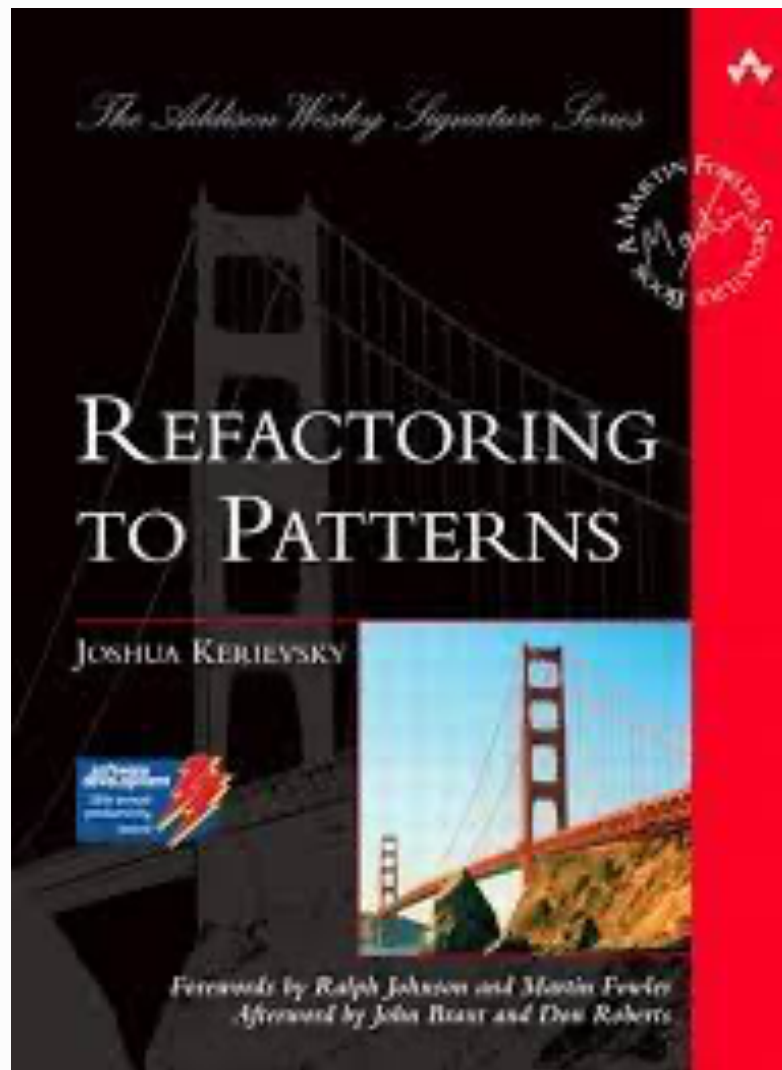
How can we manage Technical Debt?

To understand and observe technical debt we can observe code smells.

Code smells are **symptoms** that can help to perceive the presence of suboptimal design practices [3].

Example: Long Method

```
public String statement() {  
    double totalAmount = 0;  
    int frequentRenterPoints = 0;  
    Enumeration rentals = _rentals.elements();  
    String result = "Rental Record for " + getName() + "\n";  
    while (rentals.hasMoreElements()) {  
        Rental each = (Rental) rentals.nextElement();  
  
        //determine amounts for each line  
        double thisAmount = amountFor(each);  
  
        //add frequent renter points  
        if (each.getMovie().getPriceCode() == Movie.NEW_RELEASE &&  
            each.getDaysRented() > 1)  
            frequentRenterPoints += 2;  
        else  
            frequentRenterPoints++;  
  
        //show figures for this rental  
        result += "\t" + each.getMovie().getTitle() + "\t" +  
            String.valueOf(thisAmount) + "\n";  
        totalAmount += thisAmount;  
    }  
  
    //add footer lines  
    result += "Amount owed is " + String.valueOf(totalAmount) + "\n";  
    result += "You earned " + String.valueOf(frequentRenterPoints) +  
        " frequent renter points";  
    return result;  
}
```



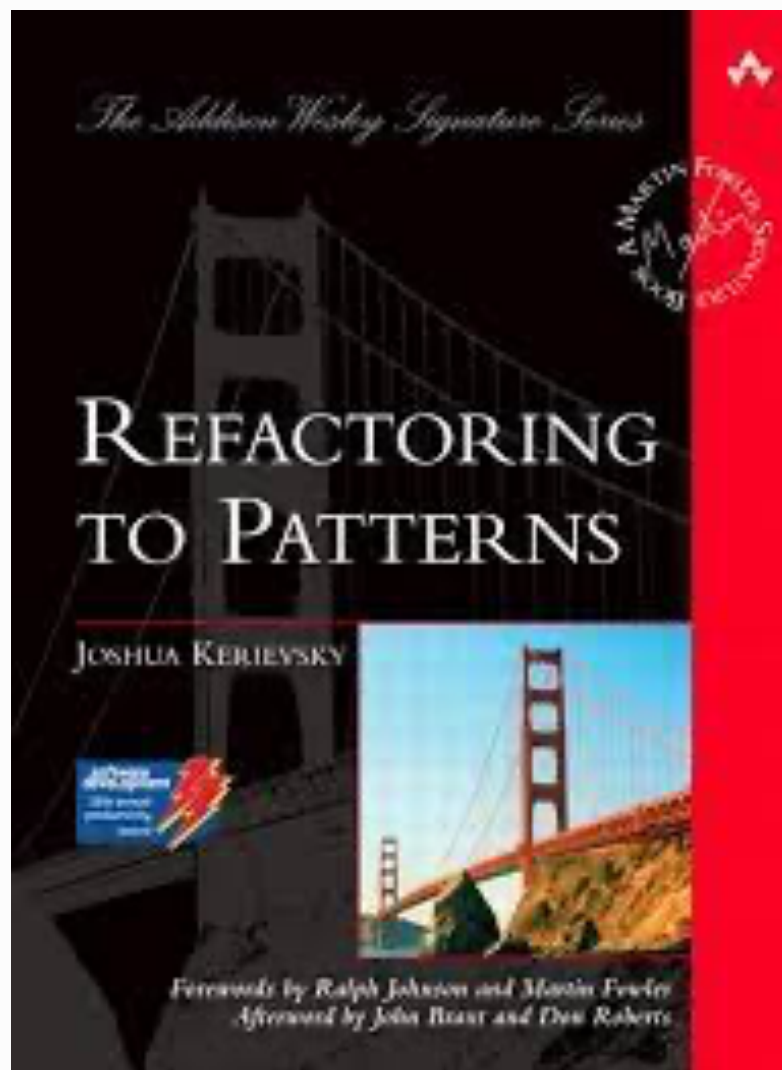
How can we manage Technical Debt?

To understand and observe technical debt we can observe code smells.

Code smells are **symptoms** that can help to perceive the presence of suboptimal design practices [3].

Example: Long Method

```
public String statement() {  
    double totalAmount = 0;  
    int frequentRenterPoints = 0;  
    Enumeration rentals = _rentals.elements();  
    String result = "Rental Record for " + getName() + "\n";  
    while (rentals.hasMoreElements()) {  
        Rental each = (Rental) rentals.nextElement();  
  
        //determine amounts for each line  
        double thisAmount = amountFor(each);  
  
        //add frequent renter points  
        if (each.getMovie().getPriceCode() == Movie.NEW_RELEASE &&  
            each.getDaysRented() > 1)  
            frequentRenterPoints += 2;  
        else  
            frequentRenterPoints++;  
  
        //show figures for this rental  
        result += "\t" + each.getMovie().getTitle() + "\t" +  
            String.valueOf(thisAmount) + "\n";  
        totalAmount += thisAmount;  
    }  
  
    //add footer lines  
    result += "Amount owed is " + String.valueOf(totalAmount) + "\n";  
    result += "You earned " + String.valueOf(frequentRenterPoints) +  
        " frequent renter points";  
    return result;  
}
```



[3] Fowler, Martin. Refactoring: improving the design of existing code. Addison-Wesley Professional, 2018.

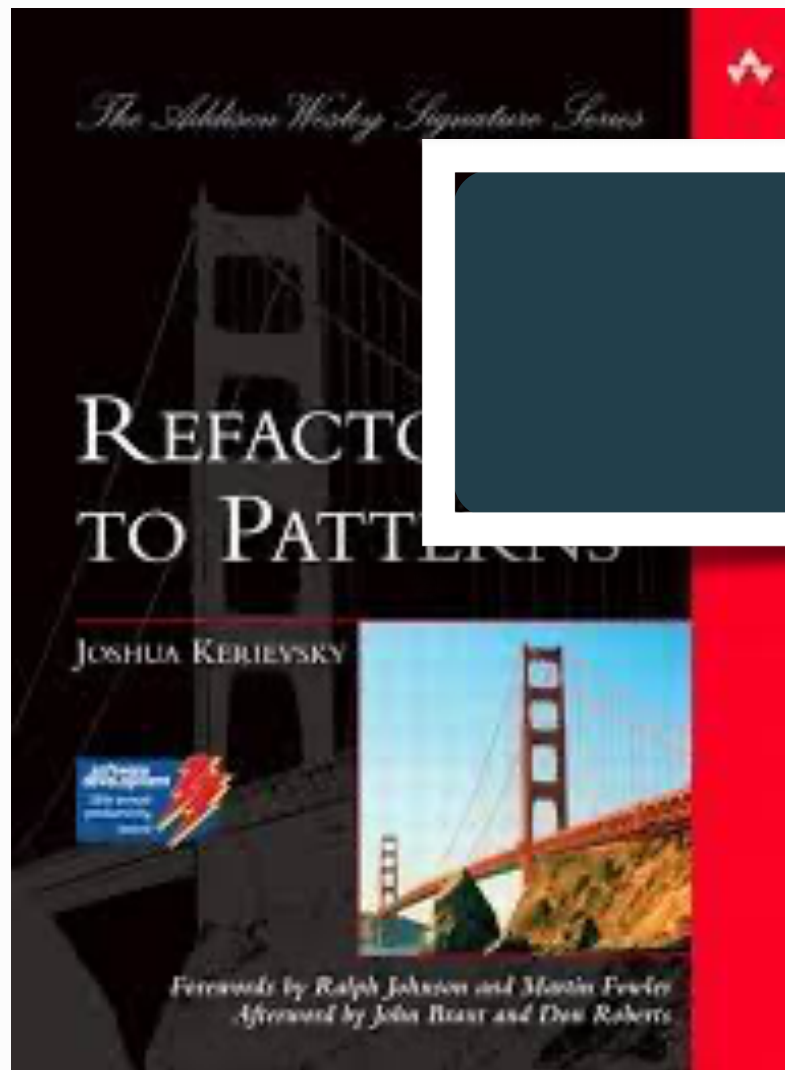
How can we manage Technical Debt?

To understand and observe technical debt we can observe code smells.

Code smells are **symptoms** that can help to perceive the presence of suboptimal design practices [3].

Example: Long Method

Code Smells are not bugs!



```
frequentRenterPoints += 2;
else
    frequentRenterPoints++;

//show figures for this rental
result += "\t" + each.getMovie().getTitle() + "\t" +
String.valueOf(thisAmount) + "\n";
totalAmount += thisAmount;
}

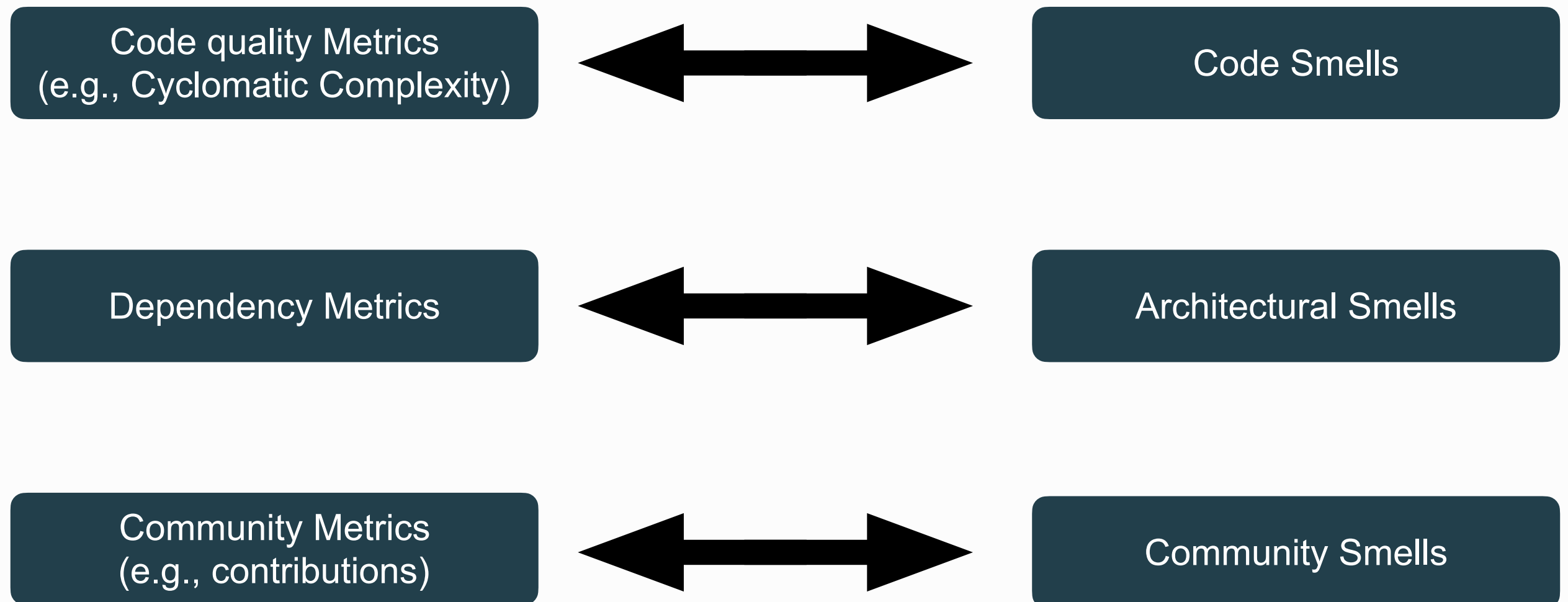
//add footer lines
result += "Amount owed is " + String.valueOf(totalAmount) + "\n";
result += "You earned " + String.valueOf(frequentRenterPoints) +
" frequent renter points";
return result;
}
```

[3] Fowler, Martin. Refactoring: improving the design of existing code. Addison-Wesley Professional, 2018.

How can we manage Technical Debt?

Smells are way simpler to detect respect to other types of quality issues, since they are visible in the code.

Along with other metrics they are used also to **estimate** technical debt.



How can we manage Technical Debt?

Through static analysis tools and code quality metrics, we can detect these issues and state the technical debt.

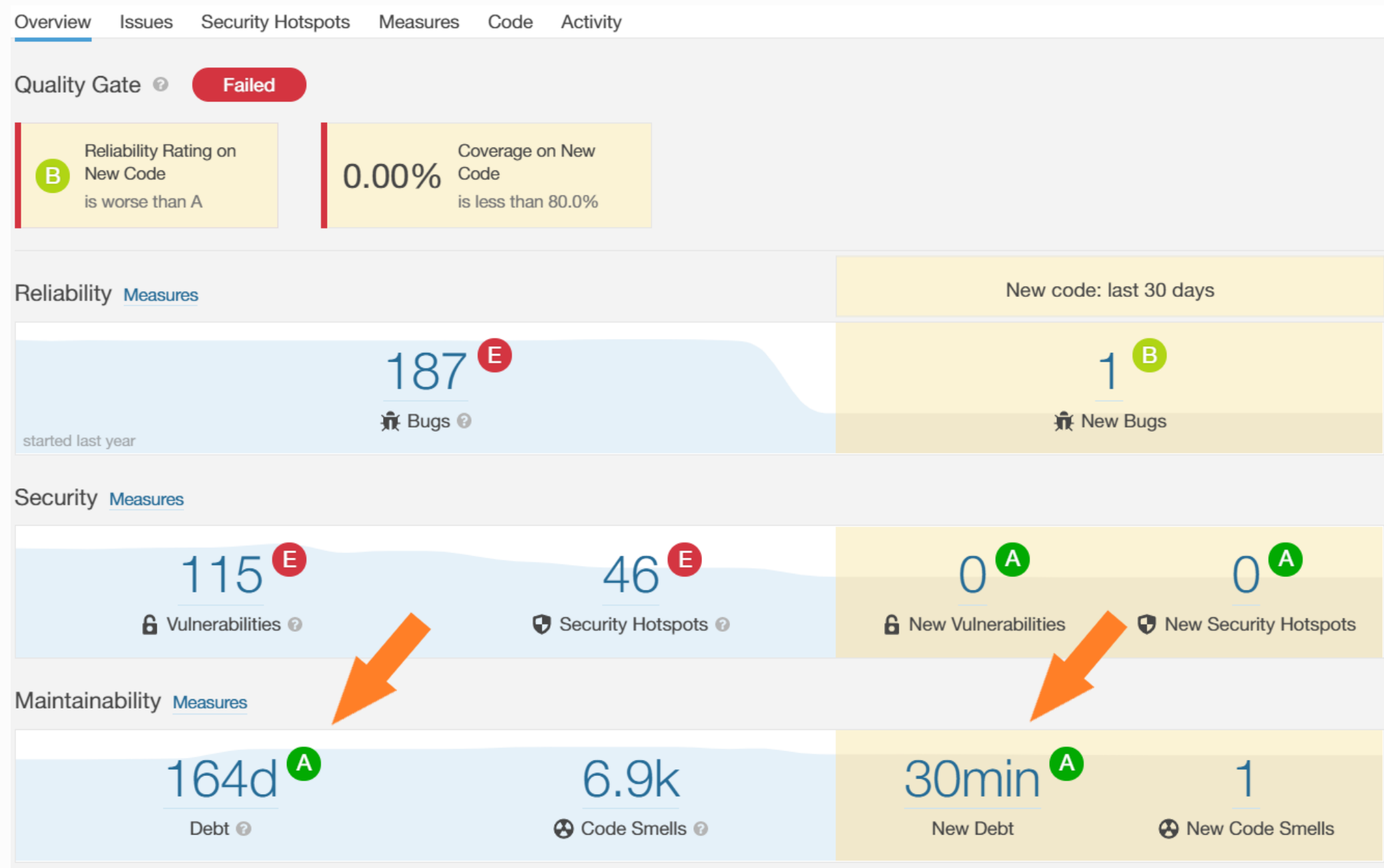
The screenshot shows the SonarQube interface with the 'Rules' tab selected. The left sidebar contains filters for Language (Java, 194), Type (Bug: 100, Vulnerability: 33, Code Smell: 194, Security Hotspot: 6), Tag, Repository, Default Severity, Status, Available Since, and Template. The main area displays a list of 194 rules, with the first 10 visible. Each rule entry includes a status icon, a description, the language (Java), the rule type (Code Smell), and associated tags. Navigation controls at the top right of the rules list allow for selecting rules and navigating between them.

Rule ID	Description	Language	Rule Type	Tags
S1133	"@Deprecated" code should not be used	Java	Code Smell	cert, cwe, obsol...
S1134	"@Override" should be used on overriding and implementing methods	Java	Code Smell	bad-practice
S1135	"Arrays.stream" should be used for primitive arrays	Java	Code Smell	performance
S1136	"catch" clauses should do more than rethrow	Java	Code Smell	clumsy, finding, ...
S1137	"clone" should not be overridden	Java	Code Smell	suspicious
S1138	"Cloneables" should implement "clone"	Java	Code Smell	api-design, con...
S1139	"close()" calls should not be redundant	Java	Code Smell	redundant
S1140	"Collections.EMPTY_LIST", "EMPTY_MAP", and "EMPTY_SET" should not be used	Java	Code Smell	obsolete, pitfall
S1141	"DateUtils.truncate" from Apache Commons Lang library should not be used	Java	Code Smell	java8, performa...
S1142	"default" clauses should be last	Java	Code Smell	misra
S1143	"entrySet()" should be iterated when both the key and value are needed	Java	Code Smell	performance
S1144	"equals" method parameters should not be marked "@Nonnull"	Java	Code Smell	

How can we manage Technical Debt?

Once we have the set of this issues, we can measure the **technical debt ratio (TDR)**.

$$\text{TDR} = (\text{Remediation Cost} / \text{Development Cost}) * 100$$



How can we manage Technical Debt?

Once we have the set of this issues, we can measure the **technical debt ratio (TDR)**.

$$\textbf{TDR} = (\text{Remediation Cost} / \text{Development Cost}) * 100$$

Golden Rule:

TDR < 5%: A good ratio, indicating a healthy project.

5% < TDR > 10%: A warning zone, suggesting the team is spending a significant amount of time on maintenance.

TDR > 20%: A high-risk level, where the team may be spending most of its time on cleanup and refactoring rather than new development.

How can we manage Technical Debt?

Now that we know how to identify TD and measure TDR, we can continue to mature our process for Tracking TD.

Maturity States:

Unaware

There is no awareness of TD in the organisation.

Ad-Hoc

Driven by individual initiative.

Systematic

Iterative process to monitor TD.

Fully-Automated

Automated data driven decision to resolve TD.



WARNING

**IF YOU DON'T SCHEDULE
REGULAR SYSTEM
MAINTENANCE,
YOUR EQUIPMENT WILL
SCHEDULE IT FOR YOU.**

Sustainable Software Engineering - Total Cost Ownership & Technical Debt

